

FIT2099 Assignment 2

Design Rationale - REQ2: Teleportation

A. Implementation of Teleportable Grounds/Items Abstraction

Design 1: Create an Abstract Class TeleportDevice

Pros:

- Shared common behaviour (destination validation, actor movement) implemented once, inherited by all devices.
- Follow DRY principle, reduced code duplication.
- Single inheritance hierarchy provides clear class relationships.

Cons:

- TeleportationTube and MagicCircle extend Ground, while AlienCube extends Item.
- Java does not support multiple inheritance, so cannot have all three extend a common TeleportDevice abstract class without breaking the Ground and Item hierarchy.
- Would require awkward multiple abstraction layers.
- Tight coupling between device implementations and abstract class; changes to abstract class cascade to all subclasses.

Design 2: Create Teleportable Interface

Pros:

- Each device can provide different destination logic without forcing common implementation.
- Allows both Ground subclasses (Tube, Circle) and Item subclasses (Cube) to be teleportable without breaking their inheritance hierarchy.
- Java supports multiple interfaces, so devices can implement Teleportable alongside their existing inheritance.
- Minimal coupling, TeleportAction depends only on Teleportable abstraction, not concrete classes.
- When new devices are added, no modification to existing code needed (Open-Closed Principle).

Cons:

- No shared attributes or behavior; each device implements full interface independently.
- Some code duplication across devices.
- Multiple classes must implement the same methods.

Chosen Design: Design 2 - Teleportable Interface

Design 2 is chosen because TeleportationTube and MagicCircle extend Ground, while AlienCube extends Item. Java does not support multiple inheritance, so Design 1 is infeasible. The interface approach resolves this by allowing all devices to implement Teleportable regardless of their base class. This adheres to DIP, TeleportAction depends on the abstraction, not concrete classes and enables OCP, for adding new devices without modification. Code duplication in side effect logic is minimal and acceptable given the distinct nature of effects.

B. Implementation of Teleportation Side Effects

Design 1: Centralized Effect System

Pros:

- Centralized control of all side effects in one place.
- Reduces code duplication, location iteration and tile modification logic written once.
- Single point of change if side effect mechanics need to be modified globally.
- Easy to add logging and debugging for all effects.

Cons:

- TeleportAction becomes coupled to EffectManager.
- Requires TeleportAction to know about device types and pass them to EffectManager.
- New device types must register their effects externally in EffectManager.
- Violates Single Responsibility Principle, device behaviour should not be scattered across different classes.
- Over-engineering for current feature scope.

Design 2: Device-Specific onTeleport() Method

Pros:

- Each device encapsulates its own behaviour, TeleportationTube handles fire, AlienCube handles toxic waste.
- TeleportAction remains simple and stateless, it only coordinates movement and calls onTeleport().
- Device-specific logic is self-contained and easy to understand.
- No coupling between TeleportAction and effect types.
- Adheres to Single Responsibility Principle, which each device is responsible for its own behaviour.
- Supports Open-Closed Principle, new devices can add new side effects without modifying existing code.

Cons:

- Some code duplication in adjacent tile iteration logic across devices.
- Each device must independently handle edge cases.
- If many devices share similar effects, duplication accumulates.

Chosen Design: Design 2 - Device-Specific onTeleport()

Design 2 is chosen because each device encapsulates its own side effect logic, adhering to SRP, device behaviour is self-contained and easy to understand. While there is some code duplication in adjacent tile iteration, it is minimal and localized, and the logic differs between them. This design also supports OCP principle, as new devices can add new side effects without modifying existing code.

C. Implementation of Door Security System

Design 1: Interface-Based Door Implementation

Pros:

- Maximum flexibility, each door independently defines its behaviour.
- Easy to add new door types without affecting existing ones.

Cons:

- Massive code duplication, every door must reimplement common logic
- No shared behaviour means every door reimplements the same unlock, lock, canActorEnter mechanics.
- Violates DRY principle.
- No common base for type checking or polymorphic operations.
- UnlockDoorAction would need to duplicate access level checking for each door type or use instanceof checks.
- Harder to maintain, changes to unlock logic must be propagated manually to all door types.

Design 2: Abstract Door Class with Concrete Implementations

Pros:

- DRY principle, common state and behaviour implemented once in abstract class, inherited by all doors.
- Reduces code duplication significantly.
- Polymorphism allows UnlockDoorAction to treat all doors uniformly via Door type, without instanceof checks or type-specific logic.
- New door types can be added easily by extending Door and implementing two methods.
- Consistent interface across all door types.
- Clear inheritance hierarchy representing.

Cons:

- Tight coupling between Door and subclasses; changes to abstract class can cascade to subclasses.
- Each concrete door must override onUnlock() even if behavior is empty.
- If future feature requires non-door Unlockable objects, abstract Door is not suitable.

Chosen Design: Design 2 - Abstract Door Class

Design 2 is chosen because the abstract Door class balances code reuse and extensibility. Three door types share unlock and lock mechanics but differ in access requirements and effects. This design adheres to DRY (common logic defined once), SRP (each door owns its own effect), and OCP (new doors added by extending, no modification to existing code). UnlockDoorAction depends on Door abstraction via DIP, avoiding instanceof checks.

D. Implementation of Access Level Management

Design 1: Store Access Level Directly on Actor

Pros:

- Direct and fast, no inventory search needed.
- Simple implementation, access level is a single attribute.
- Faster lookup $O(1)$ compared to inventory search.

Cons:

- Breaks encapsulation, access level should be granted by possessing a card, not inherent to the actor.
- Cannot represent multiple cards with different levels or support workers carrying cards they haven't "activated".
- Violates Single Responsibility Principle as actor should not directly manage security clearance, while security should be external/equipment based.
- Inflexible for future features like trading cards or card-based permissions.

Design 2: Store Access Level on AccessCard Item

Pros:

- Maintains encapsulation, access level is granted by possessing a card, not inherent to the actor.
- Semantically correct, aligns with real-world permissions model.
- Flexible for future features like trading cards, losing cards during capture, multiple cards with different levels.
- Supports Single Responsibility Principle, Inventory holds cards while UnlockDoorAction validates access.
- Card-based system is intuitive and extensible.
- Natural integration with existing inventory system.

Cons:

- Inventory search is $O(n)$ where n is inventory size.
- Less efficient than $O(1)$ actor property lookup.
- Requires additional logic in `UnlockDoorAction` to find and validate card.
- Must handle gracefully when no card is found (return failure message).

Chosen Design: Design 2 - AccessCard-Based

Design 2 chosen because access is semantically granted by "possessing" a card, not inherent to the actor. Card-based design maintains encapsulation and SRP, as inventory holds cards and `UnlockDoorAction` validates access. It supports future features (trading cards, card loss, multiple levels) while keeping systems loosely coupled via DIP.

E. Implementation of Malfunction Mechanic (50% Failure Rate)

Design 1: Malfunction as Strategy Pattern

Pros:

- Are flexible as different teleportation behaviours abstracted as strategies.
- Easy to add new teleportation types by creating new strategy classes.
- Testable, each strategy can be tested in isolation.
- Separates malfunction logic from device state.

Cons:

- Over-engineering for current requirements.
- Only one malfunction behavior exists; Strategy pattern adds unnecessary abstraction layers and classes.
- Adds indirection, understanding the teleportation flow requires following the strategy abstraction.

Design 2: Inline Random Logic in `onTeleport()`

Pros:

- Simple and direct, malfunction logic is straightforward and easy to understand.
- Self-contained in `onTeleport()`, all device behaviour in one place.
- No additional abstraction layers or classes.
- Follows current device pattern, each device handles its own logic.
- No overhead from strategy pattern indirection.
- Easy to read and maintain for current requirements.

Cons:

- Tight coupling of malfunction logic to effect execution.
- If malfunction probability changes per difficulty level (future feature), must modify `onTeleport()`.
- Testing randomness requires mocking `Random` class.

Chosen Design: Design 2 - Inline Random Logic

Design 2 chosen because the malfunction feature is a simple 50/50 probability check, not a complex behaviour abstraction. Applying Strategy pattern would violate YAGNI by over-engineering for a single use case. Inline logic naturally belongs in `onTeleport()` as a device-specific behaviour and keeps the code simple and maintainable. If future requirements add difficulty-based rates or multiple failure modes, extraction can be refactored at that time.

Summary of Design Decisions and SOLID principles

REQ2 design makes five key architectural decisions:

Teleportable Grounds/Items Abstraction uses the Teleportable Interface to support both Ground and Item inheritance while avoiding Java's multiple inheritance limitation.

Side Effects are handled by device-specific `onTeleport()` methods that encapsulate behaviour and adhere to SRP with minimal acceptable duplication.

Door Security employs an Abstract Door Class to enable code reuse while maintaining extensibility via polymorphic design and OCP.

Access Levels are stored on AccessCard Items to maintain encapsulation, ensure semantic correctness, and support future features via SRP.

Malfunction uses Inline Random Logic for simple, direct implementation following YAGNI while avoiding premature abstraction.